

Αρχές Γλωσσών Προγραμματισμού

Εργασία 3

Σκοπός αυτής της εργασίας είναι η υλοποίηση μίας γλώσσας χαμηλού επιπέδου η οποία εκτελείται σε μια μηχανή Turing. Η υλοποίηση θα απαρτίζεται από τον parser καθώς και από ένα σύστημα εκτέλεσης των μηχανών.

Turing Machines

Η μηχανή Turing είναι ένα αφηρημένο μαθηματικό μοντέλο που προτάθηκε από τον Alan Turing το 1936. Περιγράφει μία θεωρητική μηχανή η οποία αλλάζει σύμβολα πάνω σε άπειρη ταινία σύμφωνα με ένα πίνακα κανόνων. Ενώ φαίνεται να είναι μία υπεραπλοποιημένη μορφή ενός «υπολογιστή», μπορεί να περιγράψει και να προσομοιώσει κάθε αλγόριθμο.

Αποτελείται από τα εξής βασικά στοιχεία:

- **Αλφάβητο:** Το αλφάβητο Σ είναι ένα πεπερασμένο σύνολο συμβόλων, έχει πάντα το κενό σύμβολο, π.χ. $(\sqcup, \alpha, \beta, \gamma)$, όπου $\sqcup$ το κενό σύμβολο. Το αλφάβητο καθορίζει τι επεξεργάζεται η μηχανή, το οποίο είναι κάθε συνδυασμός του αλφαβήτου. Ακόμη στο αλφάβητο **δεν** περιέχονται δύο χαρακτήρες: $\leftarrow$, $\rightarrow$, οι οποίοι συμβολίζουν την κίνηση της κεφαλής μια θέση αριστερά και δεξιά αντίστοιχα.
- **Ταινία:** Η «μνήμη» της μηχανής, μια άπειρη (θεωρητικά) ταινία χωρισμένη σε κελιά. Κάθε κελί περιέχει ένα σύμβολο από το αλφάβητο Σ . Θεωρούμε ότι στην αρχή η ταινία έχει σε κάθε θέση το κενό σύμβολο.
- **Κεφαλή:** Το κελί της ταινίας που «δείχνει» στο τρέχον βήμα η μηχανή. Διαβάζει/αλλάζει το σύμβολο του κελιού και μπορεί να κινηθεί αριστερά ή δεξιά.
- **Σύνολο καταστάσεων:** Κάθε μηχανή λειτουργεί με ένα σύστημα πεπερασμένων καταστάσεων K (π.χ. $\{s, q_0, q_1, q_2\}$), το οποίο χρησιμοποιείται για να οριστεί τι θα κάνει η μηχανή σε κάθε περίπτωση. Η μηχανή αποθηκεύει σε ένα καταχωρητή την τρέχουσα κατάσταση.
- **Αρχική κατάσταση:** Η αρχική κατάσταση s ανήκει στο K και είναι η τρέχουσα κατάσταση όταν ξεκινάει μια μηχανή.

- **Τερματικές καταστάσεις:** Οι τερματικές καταστάσεις σχηματίζουν ένα σύνολο $H \subseteq K$. Όταν η μηχανή βρεθεί σε οποιαδήποτε από αυτές τις καταστάσεις τερματίζει.
- **Συνάρτηση Μετάβασης:** Η συνάρτηση μετάβασης δ είναι το «σύνολο εντολών» της μηχανής και έχει τύπο $\delta : (K \setminus H) \times \Sigma \rightarrow K \times (\Sigma \cup \{\leftarrow, \rightarrow\})$. Λαμβάνει ως ορίσματα την τρέχουσα κατάσταση και τον χαρακτήρα στο κελί που δείχνει η κεφαλή. Βάσει των ορισμάτων παράγει την νέα κατάσταση καθώς και το αν θα μετακινηθεί η κεφαλή ή αν θα γράψει και ποιον χαρακτήρα. Σε περίπτωση που είναι στο πρώτο κελί η κεφαλή και το αποτέλεσμα της συνάρτησης μετάβασης είναι ο χαρακτήρας $\leftarrow$ δεν γίνεται τίποτα παρά μόνο μετάβαση στη νέα κατάσταση.

Σημείωση: Θεωρητικά οι Ταινίες στις Μηχανές Turing ξεκινούν πάντα με ένα σύμβολο που δεικτοδοτεί την αρχή της μηχανής, αλλά για λόγους απλοποίησης έχει παραληφθεί από τη δική μας περιγραφή.

Recursive Turing Machines

Οι αναδρομικές μηχανές Turing είναι μια επέκταση αυτών που μόλις παρουσιάστηκαν. Στη δική μας γλώσσα θα χειριζόμαστε σε κάθε πρόγραμμα πολλαπλές μηχανές, που θα μοιράζονται την ταινία, την κεφαλή, και το αλφάβητο. Η βασική αλλαγή είναι ότι η συνάρτηση μετάβασης πέρα από το να γράψει στην ταινία και να μετακινήσει την κεφαλή μπορεί να καλέσει μια μηχανή, ακόμη και τον εαυτό της, και να ορίσει μια κατάσταση επιστροφής από την οποία θα συνεχίσει η μηχανή αφότου η κληθείσα τερματίσει. Κάθε μηχανή θα έχει δικό της σύνολο καταστάσεων και κατά συνέπεια αρχική κατάσταση, τελική κατάσταση και συνάρτηση μετάβασης. Όπως, κάθε μηχανή έχει αρχική κατάσταση έτσι πρέπει να υπάρχει αρχική μηχανή που θα ονομάζεται start. Πρόσθετα είναι αναγκαίο να αποθηκεύουμε την τρέχουσα μηχανή πέρα από την τρέχουσα κατάσταση.

Παράδειγμα προγράμματος

Παρατίθεται ένα παράδειγμα προγράμματος (ο χαρακτήρας `_` αντιστοιχεί στο κενό σύμβολο):

```
alphabet = {h, i}
```

```
machine clear_tape = {  
    states = {s, c1, c2, c3, e}  
    init_state = {s}  
    halting_states = {e}  
    function = {  
        s _ -> c1 w h;  
        s h -> c1 w h;  
        s i -> c1 w h;  
        c1 _ -> c2 g right;  
        c1 h -> c2 g right;  
        c1 i -> c2 g right;  
        c2 _ -> c3 g left;  
        c2 h -> c1 w _;  
        c2 i -> c1 w _;  
        c3 _ -> c3 g left;  
        c3 h -> e w _;  
        c3 i -> e w _;  
    }  
}
```

```
machine print_hi = {  
    states = {s, q, p, e}  
    init_state = {s}  
    halting_states = {e}  
    function = {  
        s _ -> q w h;  
        s h -> q w h;  
        s i -> q w h;  
        q _ -> p g right;  
        q h -> p g right;  
        q i -> p g right;  
        p _ -> e w i;  
        p h -> e w i;  
        p i -> e w i;  
    }  
}
```

```

machine start = {
    states = {s, q, e}
    init_state = {s}
    halting_states = {e}
    function = {
        s _ -> q c clear_tape;
        s h -> q c clear_tape;
        s i -> q c clear_tape;
        q _ -> e c print_hi;
        q h -> e c print_hi;
        q i -> e c print_hi;
    }
}

```

Το πρόγραμμα αρχικά περιγράφει ένα αλφάβητο και κατόπιν μια λίστα μηχανών. Σε κάθε μηχανή ορίζεται το σύνολο καταστάσεων, η αρχική κατάσταση και οι τελικές καταστάσεις. Τέλος ορίζεται η συνάρτηση μετάβασης, όπου σε κάθε γραμμή είναι ένας κανόνας της. Στα αριστερά έχει πρώτα την τρέχουσα κατάσταση και έπειτα τον χαρακτήρα στο κελί που δείχνει η κεφαλή. Στα δεξιά δίνεται πρώτα η επόμενη κατάσταση και μετά ακολουθεί ο ένας από τους χαρακτήρες w, g, c. Ο χαρακτήρας w υποδηλώνει ότι θα γραφτεί στην ταινία ο χαρακτήρας που ακολουθεί. Ο χαρακτήρας g υποδηλώνει ότι θα μετακινηθεί η κεφαλή δεξιά ή αριστερά ανάλογα με την οδηγία που ακολουθεί. Τέλος ο χαρακτήρας c υποδηλώνει ότι θα κληθεί η μηχανή της οποίας το όνομα ακολουθεί.

Η μηχανή clear_tape σβήνει όλους τους χαρακτήρες από το σημείο που δείχνει η κεφαλή και επαναφέρει την κεφαλή στην αρχική θέση. Αρχικά τοποθετεί ένα h για να σημαδέψει την αρχική θέση της κεφαλής. Έπειτα ακολουθεί μια επαναληπτική διαδικασία με τις καταστάσεις c1, c2 όπου όσο στα δεξιά συναντώνται χαρακτήρες διαφορετικοί από το κενό σύμβολο, αυτοί αντικαθίστανται από το κενό σύμβολο. Μόλις συναντηθεί κενός χαρακτήρας γίνεται μια επαναληπτική διαδικασία με τη κατάσταση c3 όπου πηγαίνει αριστερά μέχρι να συναντήσει μη κενό χαρακτήρα, δηλαδή το αρχικό σημάδι. Τότε σβήνει το σημάδι και τερματίζει.

Η μηχανή print_hi γράφει το χαρακτήρα h μετά πηγαίνει μια θέση δεξιά και γράφει το χαρακτήρα i.

Η μηχανή start καλεί την clear_tape για να σβήσει την είσοδο του χρήστη και αφού αυτή τερματίσει καλεί την print_hi για να τυπώσει το μήνυμα hi και κατόπιν τερματίζει η ίδια.

Parser (45%)

Αρχικά καλείστε να υλοποιήσετε ένα parser γι' αυτή τη γλώσσα. Το πρόγραμμα θα απαρτίζεται από το αλφάβητο της μηχανής καθώς και ένα σύνολο μηχανών. Η γλώσσα περιγράφεται από την παρακάτω γραμματική. Για λόγους διευκόλυνσης θεωρήστε ότι το σύμβολο `name` αναφέρεται σε μία συμβολοσειρά αλφαριθμητικών (π.χ. `1`, `b`, `abc123`, `13abc`, `1733`, `abc`, `a2bc`). Ο χαρακτήρας `_` αντιστοιχεί στο κενό σύμβολο.

```

$$\langle program \rangle ::= \langle alphabet \rangle \langle machine\_list \rangle$$

$$\langle alphabet \rangle ::= \text{alphabet} = \{ \langle character\_list \rangle \}$$

$$\langle character\_list \rangle ::= \langle name \rangle$$

$$| \quad \langle name \rangle , \langle character\_list \rangle$$

$$\langle machine\_list \rangle ::= \langle machine \rangle$$

$$| \quad \langle machine \rangle \langle machine\_list \rangle$$

$$\langle machine \rangle ::= \text{machine } \langle name \rangle = \{$$

$$\quad \text{states} = \{ \langle state\_list \rangle \}$$

$$\quad \text{init\_state} = \{ \langle state \rangle \}$$

$$\quad \text{halting\_states} = \langle halting\_state\_list \rangle$$

$$\quad \text{function} = \{ \langle function\_entry\_list \rangle \}$$

$$\quad \}$$

$$\langle state \rangle ::= \langle name \rangle$$

$$\langle state\_list \rangle ::= \langle state \rangle$$

$$| \quad \langle state \rangle , \langle state\_list \rangle$$

$$\langle halting\_state\_list \rangle ::= \{ \}$$

$$| \quad \{ \langle state\_list \rangle \}$$

$$\langle function\_entry\_list \rangle ::= \langle function\_entry \rangle ;$$

$$| \quad \langle function\_entry \rangle ; \langle function\_entry\_list \rangle$$

$$\langle function\_entry \rangle ::= \langle state \rangle \langle character \rangle \langle function\_result \rangle$$

$$\langle function\_result \rangle ::= \rightarrow \langle function\_result1 \rangle$$

$$\langle function\_result1 \rangle ::= \langle state \rangle \text{ w } \langle character \rangle$$

$$| \quad \langle state \rangle \text{ g left}$$

$$| \quad \langle state \rangle \text{ g right}$$

$$| \quad \langle state \rangle \text{ c } \langle name \rangle$$

```

Προτείνεται η χρήση Monads για το συγκεκριμένο τμήμα, και πιο συγκεκριμένα των Either και Maybe, καθώς είναι σε θέση να διαχειριστούν σχεδόν στο σύνολό τους πιθανά σφάλματα που προκύπτουν κατά τη διαδικασία. Το Maybe monad εκφράζει την αποτυχία ενός υπολογισμού. Το Either monad κάνει ακριβώς το ίδιο με τη μόνη διαφορά ότι επιτρέπει να δοθεί εξήγηση για την αποτυχία. Περισσότερες πληροφορίες για το Maybe monad μπορούν να βρεθούν στο βιβλίο Learn You a Haskell . Ενώ το συγκεκριμένο κεφάλαιο ξεκινάει μιλώντας για functors είναι δυνατόν να το ακολουθήσετε χωρίς να έχετε διαβάσει το προηγούμενο κεφάλαιο. Αν θέλετε να αποκτήσετε μια βαθύτερη κατανόηση αυτού του abstraction μπορείτε να διαβάσετε και το κεφάλαιο 11 όπου καλύπτονται οι functors.

Ακόμη σε αυτό το κομμάτι πρέπει να γίνει έλεγχος, αν οι μηχανές είναι καλώς ορισμένες, δηλαδή η συνάρτηση μετάβασης ορίζεται για κάθε κατάσταση της μηχανής (με εξαίρεση τις τερματικές), για κάθε χαρακτήρα του αλφαβήτου, καθώς και ότι δεν περιέχουν κάποιο επιπλέον στοιχείο. Επίσης πρέπει να γίνει έλεγχος ότι οι μηχανές που καλούνται υπάρχουν, όπως και ότι οι χαρακτήρες που γράφονται στην ταινία ανήκουν στο αλφάβητο. Έπειτα πρέπει να ελεγχθεί ότι η αρχική κατάσταση, καθώς και οι τελικές καταστάσεις, έχουν οριστεί μεταξύ των καταστάσεων της κάθε μηχανής. Τέλος πρέπει να ελεγχτεί ότι υπάρχει η μηχανή start καθώς από αυτήν ξεκινά η εκτέλεση καθώς και ότι κάθε μηχανή έχει μοναδικό όνομα.

Machine Representation (15%)

Για την αναπαράσταση της κάθε μηχανής καθώς και του συνόλου των μηχανών καλείστε να υλοποιήσετε μια δομή Map, με όποιο τρόπο κρίνετε κατάλληλο (π.χ. Balance BST). Η συγκεκριμένη δομή πρέπει να υποστηρίζει εισαγωγή στοιχείων καθώς και την αναζήτησή τους σε λογαριθμικό χρόνο. Αυτή η δομή είναι απαραίτητη για να πραγματοποιηθεί αποδοτικά το parsing καθώς και η εκτέλεση του προγράμματος. Η αποδοτικότητα στο parsing είναι εξαιρετικά σημαντική καθώς δεν είναι ασυνήθιστο για ένα πρόγραμμα να έχει χιλιάδες γραμμές κώδικα, και είναι μια αναγκαία διαδικασία. Στις interpreted γλώσσες όπως αυτή, είναι ακόμη πιο σημαντικό καθώς σε κάθε εκτέλεση εκτελείται ο parser και μια μη αποδοτική υλοποίηση θα ήταν σημείο συμφόρησης.

Συγκεκριμένα προτείνεται να αναπαραστήσετε κάθε μηχανή ως ένα tuple που θα περιέχει και maps, όπως ένα map για τις τερματικές καταστάσεις ή ένα map που θα έχει μέσα maps για τη συνάρτηση μετάβασης.

Για την αναπαράσταση της ταινίας μπορεί να χρησιμοποιηθούν δύο λίστες, μια πεπερασμένη που περιέχει όλα τα κελιά αριστερά της κεφαλής και μια άπειρη ή πεπερασμένη για το κελί που δείχνει ο δείκτης και τα κελιά δεξιά της κεφαλής.

Τέλος θα χρειαστείτε μια στοίβα (οι λίστες αρκούν) για να αποθηκεύονται ζεύγη μηχανής-κατάστασης, στα οποία θα επιστρέψουν οι κληθείσες μηχανές.

Program Evaluation (30%)

Η εκτέλεση του προγράμματος θα ξεκινά με την κεφαλή στην πρώτη θέση της ταινίας από την αρχική κατάσταση της μηχανής start. Σε κάθε βήμα θα μετακινείται η κεφαλή δεξιά ή αριστερά ή θα πραγματοποιείται εγγραφή χαρακτήρα στο κελί στο οποίο δείχνει η κεφαλή, είτε θα κληθεί μια μηχανή. Στην τελευταία περίπτωση θα αποθηκεύεται στη στοίβα η κατάσταση που ορίζει η συνάρτηση μετάβασης ως αυτή από την οποία θα συνεχιστεί η εκτέλεση αφού τερματιστεί η κληθείσα μηχανή, μαζί με την τρέχουσα μηχανή. Η εκτέλεση θα συνεχίζεται από την αρχική κατάσταση της κληθείσας μηχανής. Σε περίπτωση που γίνει μετακίνηση της κεφαλής ή εγγραφή χαρακτήρα και η επόμενη κατάσταση είναι τερματική: αν η στοίβα είναι κενή, το πρόγραμμα τερματίζει. Διαφορετικά αφαιρείται η κορυφή και συνεχίζεται η εκτέλεση από τη μηχανή και την κατάσταση που ορίζει η κορυφή της στοίβας. Επίσης, συνιστάται κατά την επιστροφή να ελέγχεται ότι η νέα κατάσταση δεν είναι κατάσταση τερματισμού. Στην περίπτωση που είναι αφαιρούνται οι εγγραφές της στοίβας μέχρι τον τερματισμό της μηχανής ή εύρεση κάποιας μη τερματικής κατάστασης.

Σημαντικό: Η είσοδος του χρήστη πάνω στην ταινία απαγορεύεται να περιέχει το κενό σύμβολο και θα τοποθετείται από το δεύτερο κελί και μετά.

Programs (10%)

Καλείστε να κατασκευάσετε και να παραδώσετε 2 προγράμματα, στη γλώσσα που περιγράφεται. Τα συγκεκριμένα προγράμματα δεν πρέπει να είναι τετριμμένα αλλά να αποσκοπούν να αναδείξουν τη λειτουργικότητα της υλοποίησής σας. Πρέπει να συνοδεύονται από μια σύντομη περιγραφή, στην οποία μπορούν και να τονιστούν διάφορα πιθανά πλεονεκτήματα της υλοποίησής σας, όπως οικονομία μνήμης κτλ. Αποφύγετε μεγάλα αλφάβητα και περίπλοκες ιδέες καθώς οδηγούν σε μεγάλα προγράμματα.

Παραδοτέο

Συνιστάται να οργανώσετε τον κώδικά σας σε διαφορετικά αρχεία και Modules. Η μεταγλώττιση θα πρέπει να παράγει ένα εκτελέσιμο αρχείο. Μπορείτε να χρησιμοποιήσετε μόνο τον ghc ή και το cabal build system. Στη δεύτερη περίπτωση θα πρέπει να παραδώσετε και τα αρχεία του cabal configuration μαζί με την εντολή για μεταγλώττιση και εκτέλεση.

Το εκτελέσιμο θα παίρνει ένα όρισμα από τη γραμμή εντολών, το όνομα του αρχείου με το πρόγραμμα. Έπειτα θα ζητά είσοδο από τον χρήστη, ο οποίος θα δώσει χαρακτήρες χωρισμένους με κενά. Αφού τερματίσει η μηχανή το πρόγραμμα θα τυπώσει όλους τους μη κενούς χαρακτήρες στην ταινία καθώς και την κατάσταση τερματισμού υπό την οποία τερμάτισε.

Απαγορεύεται αυστηρά η χρήση external modules και modules όπως Data.Seq, Data.Array, κτλ.

Επιτρέπονται τα modules σχετικά με Functors, Applicatives, Monads.

Καλείστε να παραδώσετε και ένα αρχείο README.md με τις σχεδιαστικές σας επιλογές και κάθε άλλο τυχόν σχόλιο.

Το παραδοτέο θα πρέπει να είναι ένα zip αρχείο με όλα τα σχετικά αρχεία.

Η εργασία μπορεί να παραδοθεί ατομικά ή από ομάδες 2 ατόμων. Στη δεύτερη περίπτωση, πρέπει να γίνει μόνο μια υποβολή.

Για περαιτέρω διευκρινίσεις, ειδικά σχετικά με πιθανά modules που μπορεί να θέλετε να χρησιμοποιήσετε, μπορείτε να απευθυνθείτε μέσω email στις παρακάτω διευθύνσεις.

Κωνσταντίνος Δημητρόπουλος	sdi2100033 [at] di.uoa.gr
Λάζαρος Χαβατζόγλου	sdi2300212 [at] di.uoa.gr